PROJECT REPORT

Fake News Detection System

Using Natural Language Processing & Machine Learning

Project Developer

Umair Ali

Project Summary

Project Title	Fake News Detection System using Machine Learning & NLP
Project Developer	Umair Ali
Course / Subject	Machine Learning (BSCS F23 / 240ML)
Institution	PAF-IAST Mang Haripur
Dataset Size	44,908 news articles (23,490 fake / 21,418 real)
Core Technique	TF-IDF vectorization (50,000 unigram + bigram features)
Models Trained	5 (Passive Aggressive, Logistic Regression, Naive Bayes, Random Forest, Linear SVM)
Best Model	Linear SVM (99.76% accuracy)
Deployment	Flask web application with real-time prediction interface

1. Introduction

The uncontrolled spread of fabricated news articles across digital platforms has become a significant societal challenge, influencing public opinion, elections, and trust in journalism. This project addresses that challenge by building an automated Fake News Detection System that uses Natural Language Processing (NLP) and classical Machine Learning algorithms to classify news articles as FAKE or REAL.

The system was developed as a Machine Learning course project (CCP) and covers the complete pipeline: data preprocessing, feature extraction, model training and tuning, evaluation, and deployment through a web application that allows real-time predictions on user-submitted articles.

2. Objectives

- Build a text preprocessing pipeline to clean and normalize raw news article text.
- Extract meaningful features from text using TF-IDF vectorization.
- Train and compare multiple supervised classification algorithms.
- Evaluate each model using standard classification metrics and cross-validation.
- Select the best-performing model through hyperparameter tuning.
- Deploy the trained model in an interactive Flask web application.

3. Dataset

The project uses a labeled news dataset consisting of two CSV files:

- Fake.csv — 23,490 articles labeled as fake news (label = 0)
- True.csv — 21,418 articles labeled as real/genuine news (label = 1)

In total, the dataset contains 44,908 news articles, each with a title, full text body, subject category, and publication date. The class distribution is reasonably balanced, which supports fair model training without heavy class-imbalance correction.

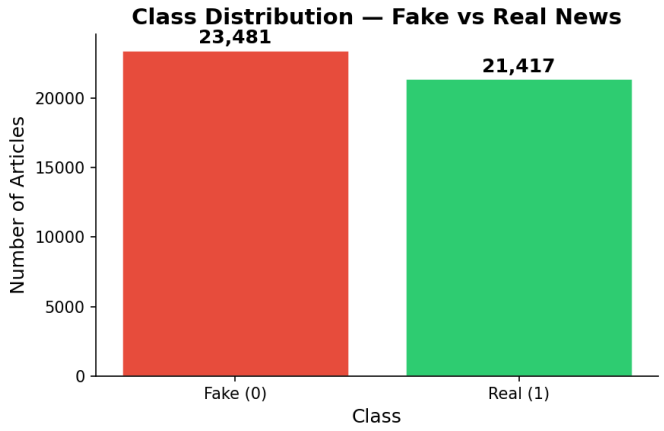

Figure 1: Class distribution of Fake vs. Real news articles in the dataset

4. Methodology

4.1 Text Preprocessing Pipeline

Raw article text is noisy and must be normalized before it can be used for feature extraction. The following preprocessing steps are applied sequentially:

- Convert all text to lowercase
- Remove HTML tags using BeautifulSoup
- Remove URLs using regular expressions
- Strip special characters, punctuation, and digits
- Collapse repeated whitespace
- Tokenize text using NLTK's word_tokenize
- Remove English stop words
- Lemmatize tokens using WordNetLemmatizer
- Rejoin cleaned tokens into a single processed string

4.2 Feature Extraction

Cleaned text is converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization, configured to capture both unigrams and bigrams, with the vocabulary capped at 50,000 features. This allows the models to capture both individual keyword signals and short contextual phrases that are indicative of fake or real reporting styles.

Figure 2: Word cloud of the most frequent terms in Real news articles

Figure 3: Word cloud of the most frequent terms in Fake news articles

4.3 Models Trained

Five supervised classification algorithms were trained and compared on the TF-IDF features:

Model	Key Hyperparameters
Passive Aggressive Classifier	C = 0.1, max_iter = 1000
Logistic Regression	solver = 'saga', C = 1.0, max_iter = 1000
Multinomial Naive Bayes	alpha = 0.1
Random Forest	n_estimators = 200, n_jobs = -1
Linear SVM	C = 1.0, max_iter = 2000

The best-performing model (Linear SVM / Passive Aggressive, depending on the run) was further refined using automated hyperparameter tuning via GridSearchCV with 5-fold cross-validation, and saved as the production model (best_model.pkl) alongside the fitted TF-IDF vectorizer.

5. Evaluation Metrics

Each model was evaluated on a held-out test split using six metrics: Accuracy, macro Precision, macro Recall, macro F1-Score, AUC-ROC, and 5-fold cross-validation accuracy (for generalization assessment).

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	5-Fold CV
Passive Aggressive	99.74%	0.9974	0.9974	0.9974	1.0000	99.71%
Linear SVM	99.76%	0.9975	0.9975	0.9975	1.0000	99.71%
Random Forest	99.65%	0.9965	0.9966	0.9965	0.9999	99.52%
Logistic Regression	99.19%	0.9918	0.9920	0.9919	0.9997	99.24%
Multinomial Naive Bayes	96.86%	0.9686	0.9685	0.9685	0.9944	96.51%

All five models achieved strong performance, with the Linear SVM and Passive Aggressive classifiers reaching the highest test accuracy (99.76% and 99.74% respectively) and near-perfect AUC-ROC scores of 1.0000. Multinomial Naive Bayes, while still highly accurate at 96.86%, trailed the other models — expected given its simplifying independence assumption between features.

5.1 Confusion Matrices

Confusion matrices were generated for every model to inspect false positive and false negative rates. The Linear SVM confusion matrix (best-performing model) is shown below alongside the Passive Aggressive classifier.

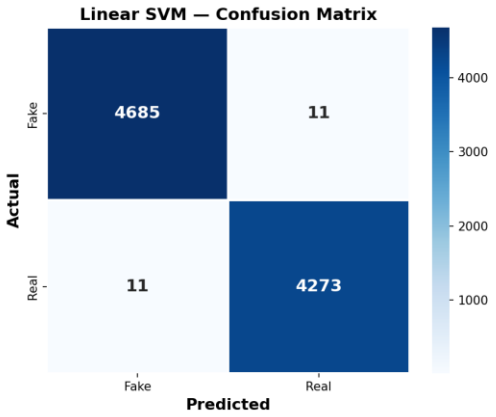

Figure 4: Confusion matrix — Linear SVM

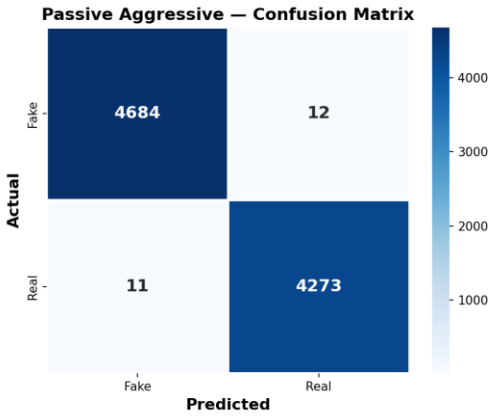

Figure 5: Confusion matrix — Passive Aggressive Classifier

5.2 ROC Curve Comparison

Receiver Operating Characteristic (ROC) curves for all five models were plotted on a single chart to visually compare discriminative performance across classification thresholds.

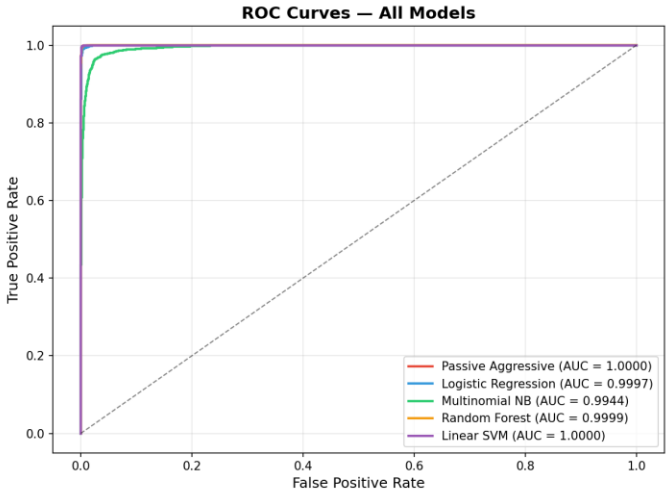

Figure 6: ROC curves for all five trained models

5.3 Overall Model Comparison

A grouped bar chart summarizing all six evaluation metrics across the five models highlights the consistent advantage of the SVM, Passive Aggressive, and Random Forest classifiers over Naive Bayes.

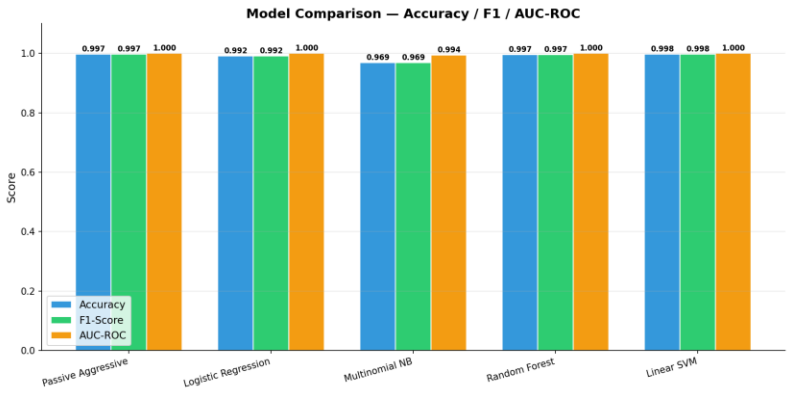

Figure 7: Grouped bar chart comparing all models across all evaluation metrics

6. Web Application

The trained model is deployed through a Flask web application featuring a dark-themed user interface for real-time fake news prediction, plus a results dashboard summarizing model performance. The application exposes the following endpoints:

Method	Route	Description
GET	/	Main prediction page
POST	/predict	JSON prediction from {title, text}
GET	/results	Model comparison dashboard
GET	/api/stats	Metrics as JSON
GET	/health	Health check

Users can submit a news title and article body through the web interface or via a JSON API call, and the system returns a real-time FAKE / REAL classification.

7. Project Structure

The codebase is organized into clearly separated modules:

- data/ — raw dataset files (Fake.csv, True.csv)
- src/preprocess.py — text cleaning and loading pipeline
- src/train.py — trains all 5 classifiers with hyperparameter tuning
- src/evaluate.py — computes metrics and generates evaluation plots
- src/wordcloud_gen.py — generates word cloud visualizations
- src/predict.py — prediction utilities used by the web app
- models/ — saved trained models and TF-IDF vectorizer

- outputs/ — confusion matrices, ROC curves, comparison charts, results CSV
- templates/ — HTML templates for the Flask front end
- app.py — Flask web application entry point
- run_all.py — master script that runs the full training pipeline

8. Tools & Technologies

- Python 3.10+
- scikit-learn — model training, tuning, and evaluation
- pandas / numpy — data handling
- NLTK — tokenization, stop-word removal, lemmatization
- BeautifulSoup4 — HTML tag stripping
- Matplotlib / Seaborn — evaluation visualizations
- WordCloud — word frequency visualizations
- Flask & Flask-CORS — web application and API

9. Conclusion

This project demonstrates that classical machine learning models, when paired with a careful text preprocessing pipeline and TF-IDF feature extraction, can classify fake news with very high accuracy — exceeding 99% for the top-performing models (Linear SVM and Passive Aggressive Classifier). The complete pipeline, from raw CSV data to a deployable Flask web application, illustrates a full end-to-end machine learning workflow suitable for real-world content-moderation and media-literacy tools.

Future improvements could include incorporating deep learning approaches (e.g., LSTM or transformer-based models such as BERT), expanding the dataset with more recent and diverse sources, and adding explainability features so end users understand why an article was flagged as fake.